

AI/ML Engineer Pre-Work Assignment

Submitted to

Navgurukul

Project

ETA Prediction for Last-Mile Delivery Optimization

Submitted By

Dipanshu Kumar Singh

GitHub: www.github.com/Dnshusngh

LinkedIn: www.linkedin.com/in/dipanshu-kumar-singh-602998228

Email: dpnshusingh@gmail.com

Table of Contents	
1.	Project Context
2.	Technical Implementation
3.	Architecture & Workflow
4.	Critical Code Walkthrough
5.	Data Flow Analysis
6.	Technical Decisions & Trade-offs
7.	Scaling Challenges & Mitigation
8.	Learning & Iteration
9.	Business Impact
10.	Conclusion

Section 1: Context (Brief)

Section 1: Context (Brief)

Project Description

This project focused on optimizing last-mile logistics operations using Artificial Intelligence (AI), Machine Learning (ML), and Generative AI (GenAI) techniques. The system was designed to improve delivery efficiency through ETA prediction, intelligent route optimization, delivery partner allocation, and spatio-temporal demand forecasting using large-scale logistics datasets containing courier trajectories, package information, road network data, traffic conditions, and operational delivery records.

The objective of the system was to reduce delivery delays, optimize courier allocation, minimize operational costs, improve ETA accuracy, and enhance customer satisfaction through scalable and data-driven logistics intelligence.

The project involved the complete machine learning lifecycle:

- Data Collection
- Data Cleaning & Preprocessing
- Exploratory Data Analysis
- Feature Engineering
- Model Training & Evaluation
- Pipeline Development
- Dashboarding & Monitoring
- Deployment Planning

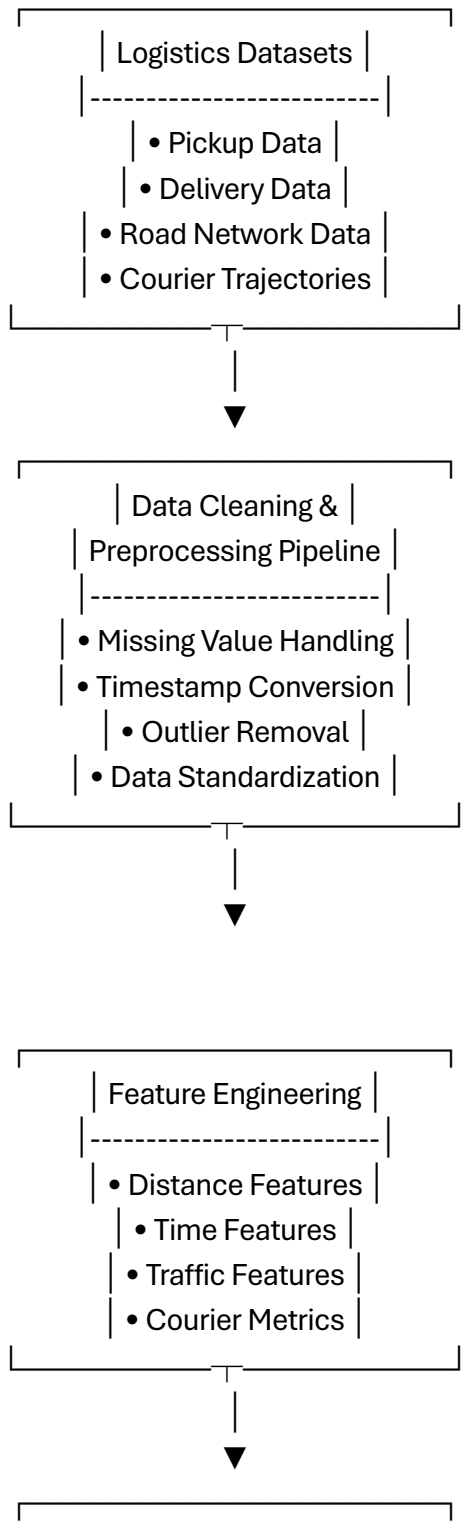
Primary Technical Constraints

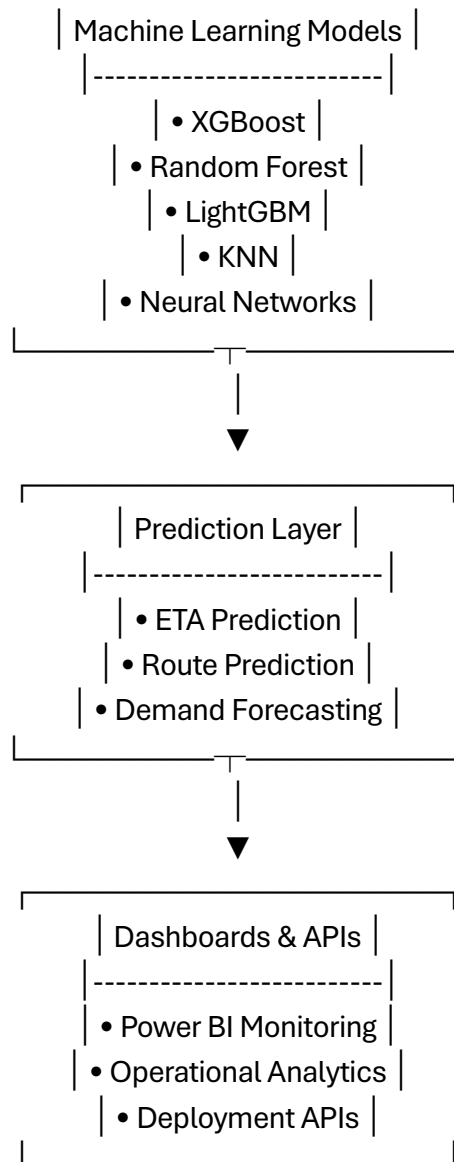
- The project involved multiple real-world technical constraints:

Constraint	Impact
Large-scale datasets (10M+ records)	Increased preprocessing and training complexity
Missing GPS coordinates	Reduced geospatial reliability
Traffic variability	Dynamic ETA fluctuations
Temporal dependencies	Risk of data leakage during training
High-cardinality courier/location data	Increased feature engineering complexity
Real-time prediction latency	Operational deployment challenges
Multi-source data integration	Data synchronization issues
Geospatial inconsistencies	Route mapping complexity

Section 2: Technical Implementation (Detailed)

System Architecture





Architecture Explanation

The system processes large-scale logistics datasets through preprocessing and feature engineering pipelines before training machine learning models for ETA prediction, route optimization, and delivery intelligence. The outputs are integrated into dashboards and deployment pipelines to support operational decision-making, delivery performance monitoring, and resource optimization.

Datasets Used

Dataset	Purpose
LaDe-P Pickup Dataset	Pickup task analysis
LaDe-D Delivery Dataset	Delivery duration prediction

Road Network Dataset	Route & traffic intelligence
Courier Trajectory Dataset	GPS movement analysis

Data Preprocessing Pipeline

The preprocessing stage focused on improving data quality and ensuring model readiness.

Preprocessing Operations

1. Missing Value Handling

- Removed records with critical timestamp gaps
- Imputed missing coordinates using regional averages
- Replaced missing categorical values with placeholders

Imputation Strategy

If <5% missing: Use median/mode replacement.

If >5% missing: Consider removing or filling with external data.

```
[ ]: import pandas as pd

def handle_missing_data(df, threshold=5):
    """
    Handles missing data in a DataFrame by imputing columns with < threshold% missing data
    and removing columns with ≥ threshold% missing data.

    """
    # Calculate total missing values for each column
    missing_data = df.isnull().sum()

    # Calculate the total number of rows
    total_rows = df.shape[0]

    # Calculate the percentage of missing values for each column
    missing_percentage = (missing_data / total_rows) * 100

    # Handle missing data
    for col in df.columns:
        pct = missing_percentage[col]

        # Impute columns with < threshold% missing data
        if 0 < pct < threshold:
            if df[col].dtype in ['float64', 'int64']:
                df[col].fillna(df[col].median(), inplace=True)
            else:
                df[col].fillna(df[col].mode()[0], inplace=True)

        # Remove columns with ≥ threshold% missing data
        elif pct >= threshold:
            df.drop(columns=[col], inplace=True)

    return df
```

2. Duplicate Removal

Used `package_id` as a unique identifier to remove duplicate delivery records.

3. Timestamp Standardization

Converted delivery timestamps into datetime format for temporal analysis.

4. Outlier Detection

Applied:

- IQR method
- Z-score filtering

for abnormal delivery durations and unrealistic geospatial records.

Timestamp Standardization

Convert timestamps to a consistent UTC or local timezone.

Extract hour, day, weekday/weekend from timestamps for later use.

```
]:
```

```
def convert_and_extract_datetime_features(df, columns):
    for col in columns:
        # Extract hour, day, and minute directly from the string
        df[f'{col}_hour'] = pd.to_datetime(df[col], format="%m-%d %H:%M:%S", errors='coerce').dt.hour
        df[f'{col}_day'] = pd.to_datetime(df[col], format="%m-%d %H:%M:%S", errors='coerce').dt.day
        df[f'{col}_weekday'] = pd.to_datetime(df[col], format="%m-%d %H:%M:%S", errors='coerce').dt.weekday
        df[f'{col}_is_weekend'] = df[f'{col}_weekday'].apply(lambda x: 1 if x >= 5 else 0)
        # 1 if Saturday or Sunday, else 0

        # Check for NaT values and print warning
        if df[f'{col}_hour'].isna().sum() > 0:
            print(f"⚠ Warning: Found NaT in extracted features for column '{col}' due to format mismatch or missing values.")

    return df

]:
```

```
# Converting datetime columns for delivery_df
delivery_df = convert_and_extract_datetime_features(delivery_df, [
    'accept_time',
    'delivery_time',
])

# Converting datetime columns for pickup_df
pickup_df = convert_and_extract_datetime_features(pickup_df, [
    'accept_time',
    'pickup_time'
```

5. Feature Normalization

Used:

- StandardScaler
- MinMaxScaler

for improving model convergence.

◆ Location Data Normalization

```
] : # Normalize location data
    delivery_df[['lng', 'lat', 'accept_gps_lng', 'accept_gps_lat', 'delivery_gps_lng', 'delivery_gps_lat']] = \
        delivery_df[['lng', 'lat', 'accept_gps_lng', 'accept_gps_lat', 'delivery_gps_lng', 'delivery_gps_lat']].astype(float)

] : pickup_df[['lng', 'lat']] = \
    pickup_df[['lng', 'lat']].astype(float)
```

Feature Engineering

Feature engineering was one of the most critical components of the project.

Time-Based Features

- Hour of delivery
- Day of week
- Weekend indicator
- Rush-hour indicator
- Task duration

ADDING ETA VARIABLE

Extracting Rush Hour

```
[ ] : current_year = 2025

[ ] : delivery_df['accept_time'] = pd.to_datetime(f"{current_year}-" + delivery_df['accept_time'])
    delivery_df['delivery_time'] = pd.to_datetime(f"{current_year}-" + delivery_df['delivery_time'])

[ ] : pickup_df['pickup_time'] = pd.to_datetime(f"{current_year}-" + pickup_df['pickup_time'])
    pickup_df['accept_time'] = pd.to_datetime(f"{current_year}-" + pickup_df['accept_time'])

[ ] : def is_rush_hour(time):
        hour = time.hour
        return ((7 <= hour < 9) or (16 <= hour < 18))

[ ] : pickup_df['is_rush_hour'] = pickup_df['accept_time'].apply(is_rush_hour)
    delivery_df['is_rush_hour'] = delivery_df['accept_time'].apply(is_rush_hour)

[ ] : # Calculate ETA for pickup (time between accept and pickup time)
    pickup_df['ETA_pickup'] = (pickup_df['pickup_time'] - pickup_df['accept_time']).dt.total_seconds() / 60 # in minutes

    # Calculate ETA for delivery (time between accept and delivery time)
    delivery_df['ETA_delivery'] = (delivery_df['delivery_time'] - delivery_df['accept_time']).dt.total_seconds() / 60 # in minutes
```

Geospatial Features

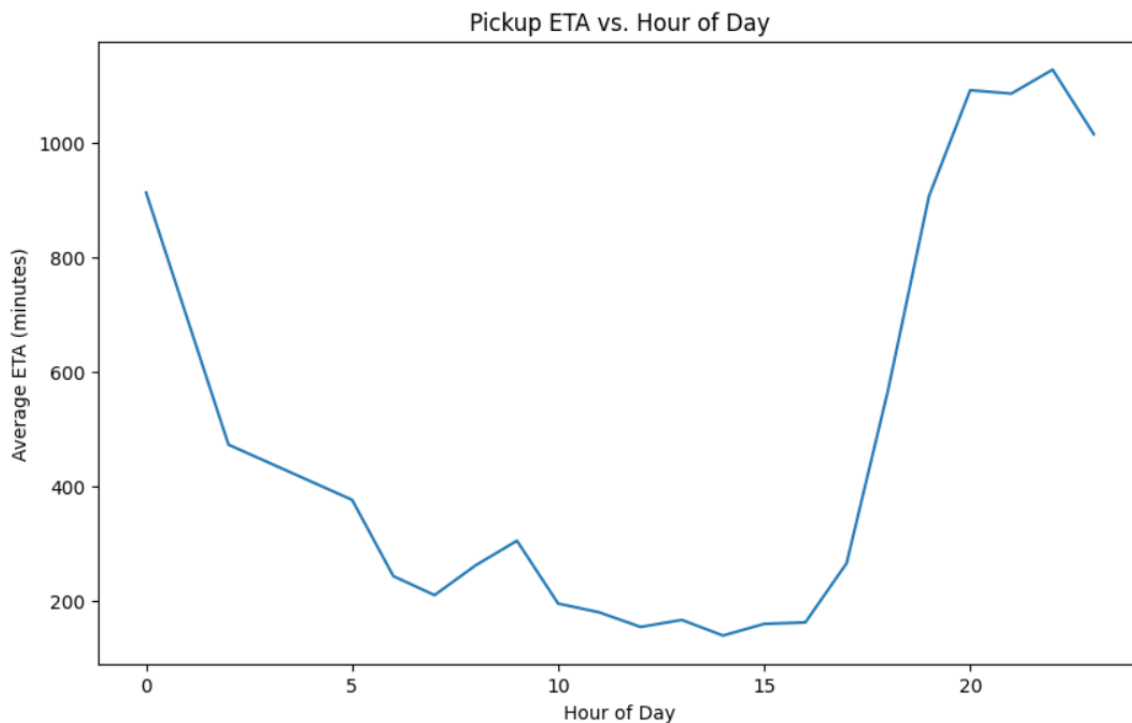
- Distance calculations
- Route density

- Traffic congestion mapping
- GPS normalization

```
import matplotlib.pyplot as plt

# Pickup ETA vs. Time of Day
plt.figure(figsize=(10, 6))
pickup_df.groupby('accept_time_hour')['ETA_pickup'].mean().plot()
plt.title("Pickup ETA vs. Hour of Day")
plt.xlabel("Hour of Day")
plt.ylabel("Average ETA (minutes)")
plt.show()

# Delivery ETA vs. Time of Day
plt.figure(figsize=(10, 6))
delivery_df.groupby('delivery_time_hour')['ETA_delivery'].mean().plot()
plt.title("Delivery ETA vs. Hour of Day")
plt.xlabel("Hour of Day")
plt.ylabel("Average ETA (minutes)")
plt.show()
```

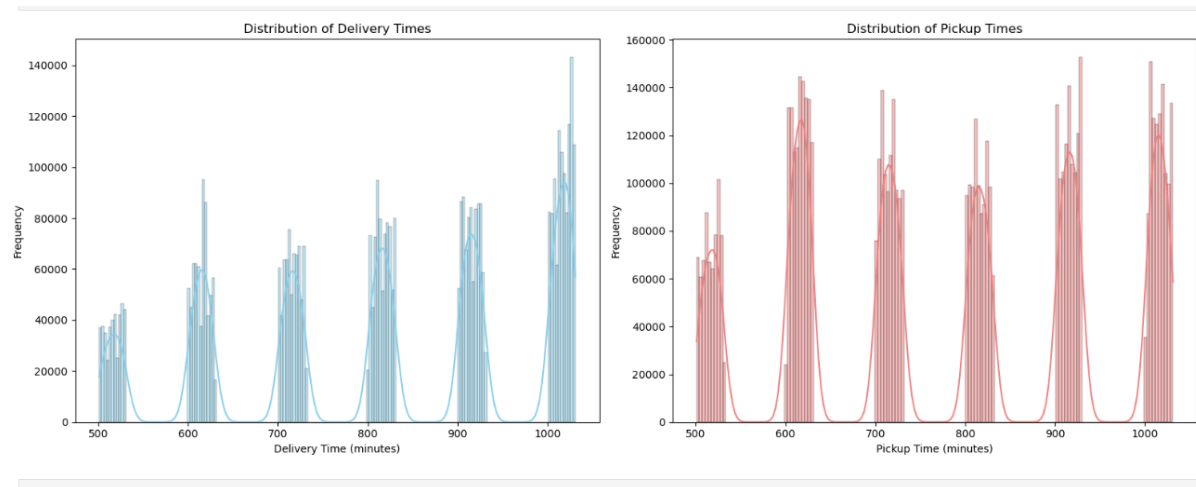



```
# Distribution of delivery times (ds) and pickup times (ds)
fig, axs = plt.subplots(1, 2, figsize=(16, 6))

# Delivery data
sns.histplot(delivery_df['ds'], kde=True, ax=axs[0], color='skyblue')
axs[0].set_title('Distribution of Delivery Times')
axs[0].set_xlabel('Delivery Time (minutes)')
axs[0].set_ylabel('Frequency')

# Pickup data
sns.histplot(pickup_df['ds'], kde=True, ax=axs[1], color='lightcoral')
axs[1].set_title('Distribution of Pickup Times')
axs[1].set_xlabel('Pickup Time (minutes)')
axs[1].set_ylabel('Frequency')

plt.tight_layout()
plt.show()
```



Operational Features

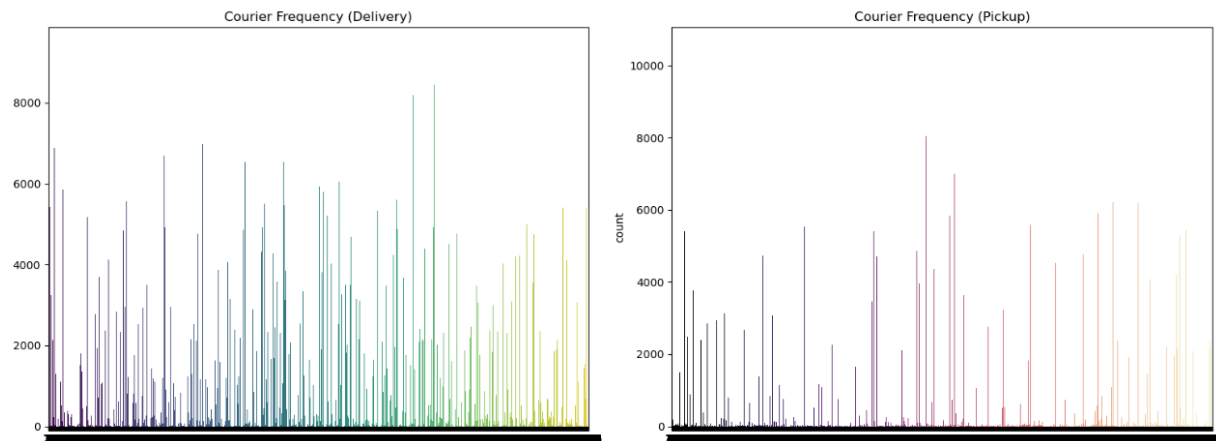
- Package volume per courier
- Delivery density per region
- Courier performance metrics
- Delivery delay frequency

```
# Courier frequency in delivery and pickup data
fig, axs = plt.subplots(1, 2, figsize=(16, 6))

# Delivery data courier frequency
sns.countplot(x='courier_id', data=delivery_df, ax=axs[0], palette='viridis')
axs[0].set_title('Courier Frequency (Delivery)')

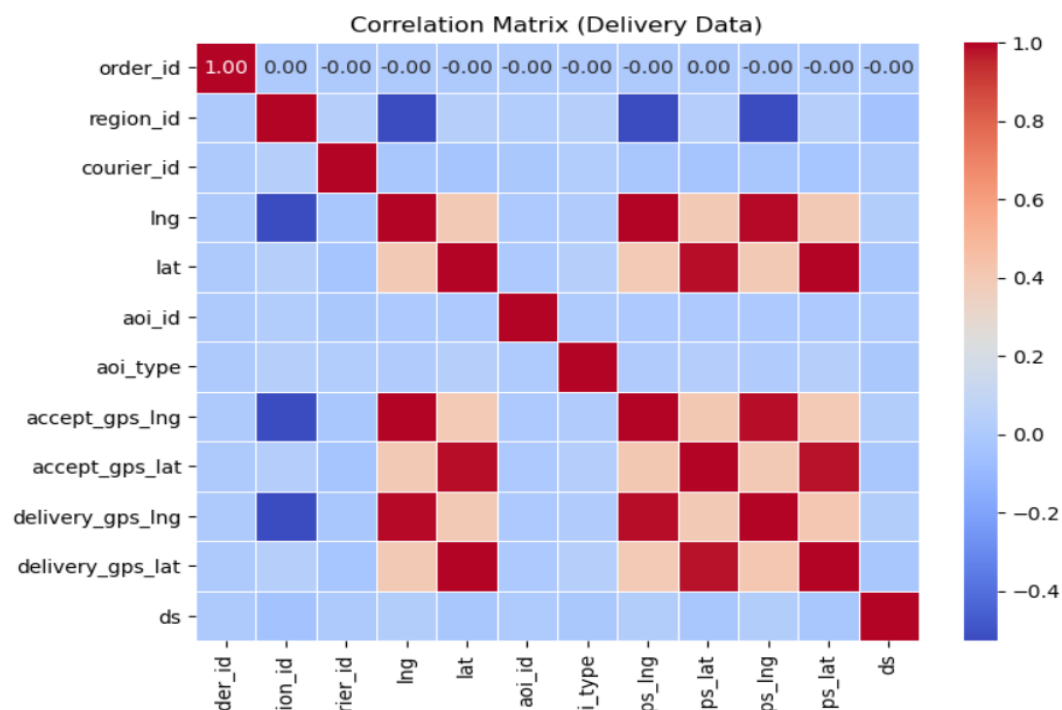
# Pickup data courier frequency
sns.countplot(x='courier_id', data=pickup_df, ax=axs[1], palette='magma')
axs[1].set_title('Courier Frequency (Pickup)')

plt.tight_layout()
plt.show()
```



```
plt.figure(figsize=(8, 6))
# Select only numeric columns for correlation
corr_matrix = delivery_df.select_dtypes(include=['float64', 'int64']).corr()

sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".2f", linewidths=0.5)
plt.title('Correlation Matrix (Delivery Data)')
plt.show()
```



```

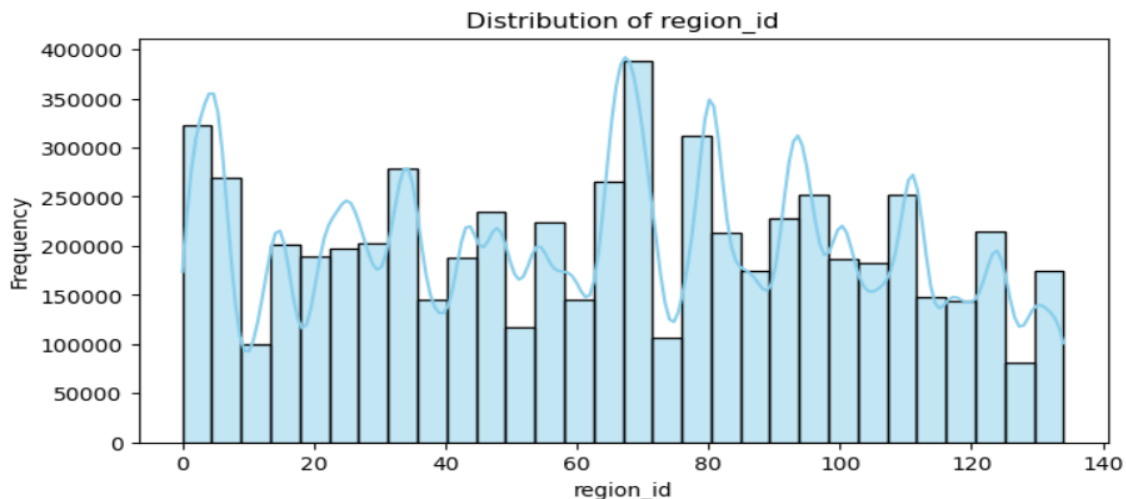
# Numerical features summary
numerical_features = pickup_df.select_dtypes(include=['float64', 'int64'])

# Summary statistics
print(numerical_features.describe())

# Histograms and KDE plots
import seaborn as sns
import matplotlib.pyplot as plt

for col in numerical_features.columns:
    plt.figure(figsize=(8, 4))
    sns.histplot(pickup_df[col], kde=True, color='skyblue', bins=30)
    plt.title(f'Distribution of {col}')
    plt.xlabel(col)
    plt.ylabel('Frequency')
    plt.show()

```



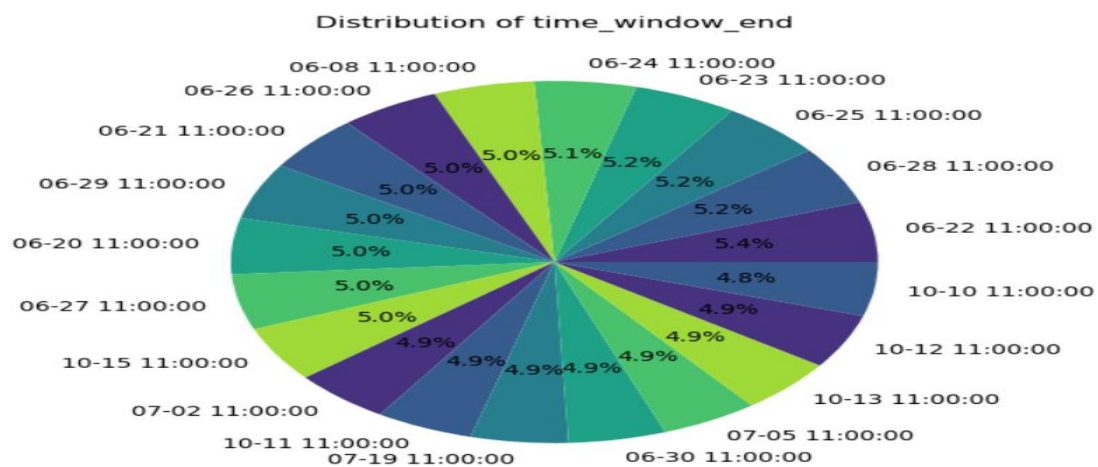
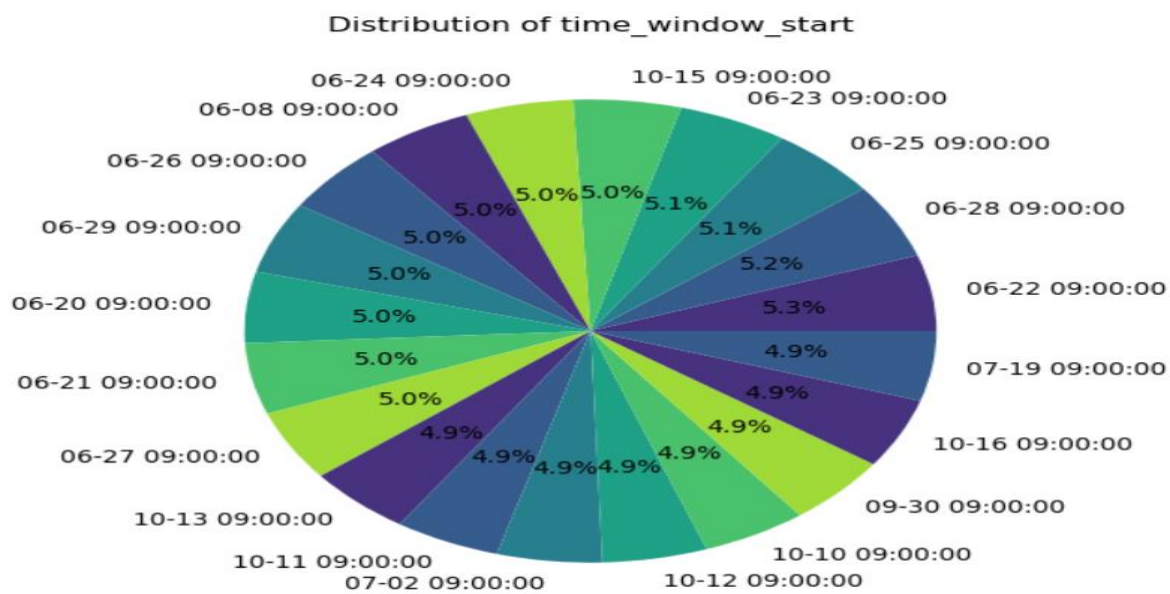
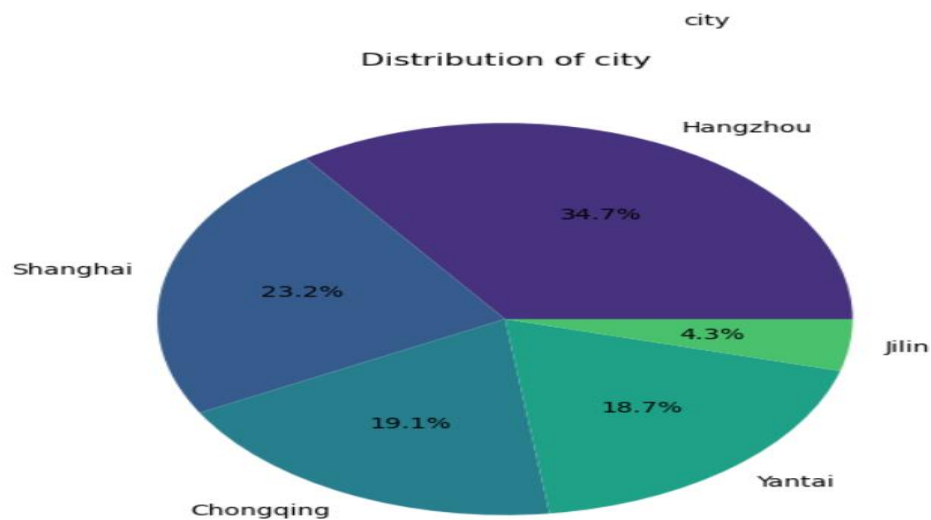
```

# Loop over categorical columns
for col in pickup_df.select_dtypes(include=['object']).columns:
    if pickup_df[col].nunique() > 20: # Limit to top 20 unique values
        top_categories = pickup_df[col].value_counts().nlargest(20).index
        data = pickup_df[pickup_df[col].isin(top_categories)]
    else:
        data = pickup_df

    plt.figure(figsize=(8, 4))
    sns.countplot(x=col, data=data, palette='viridis')
    plt.title(f'Frequency of {col}')
    plt.xticks(rotation=45)
    plt.show()

    plt.figure(figsize=(6, 6))
    data[col].value_counts().plot.pie(autopct='%1.1f%%', colors=sns.color_palette('viridis'))
    plt.title(f'Distribution of {col}')
    plt.ylabel('')
    plt.show()

```



3) Applying PCA

```
# Reduce Dimensionality using PCA (if necessary)
pca = PCA(n_components=5) # Adjust based on variance explained
principal_components_pickup = pca.fit_transform(pickup_df[num_features])

pca_df_pickup = pd.DataFrame(data=principal_components_pickup, columns=[f'PC{i+1}' for i in range(5)])

# Merge PCA Data
pickup_df = pd.concat([pickup_df, pca_df_pickup], axis=1)

# Simulating some data for demonstration (Replace this with your actual dataset)
np.random.seed(42)
pickup_df = pd.DataFrame(np.random.rand(1000, 6), columns=['lng', 'lat', 'pickup_gps_lng', 'pickup_gps_lat', 'accept_gps_lng', 'accept_gps_lat'])

# Convert data types to reduce memory usage
pickup_df = pickup_df.astype({col: 'float32' for col in pickup_df.select_dtypes('float64').columns})

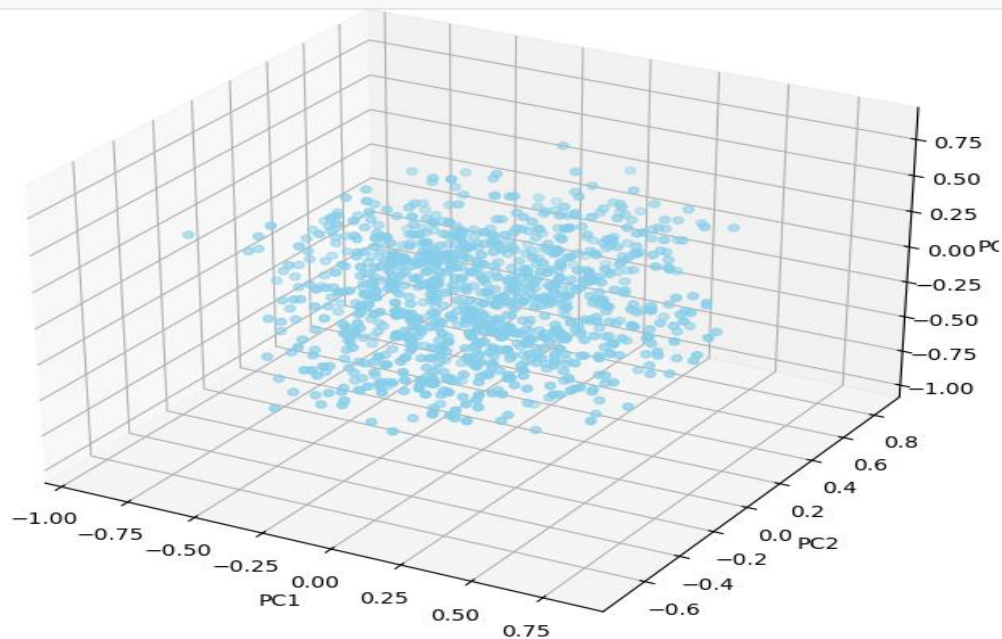
# Select numerical features
num_features = pickup_df.dropna()

# Scale Data
scaler = MaxAbsScaler()
scaled_data = scaler.fit_transform(num_features)

# Apply Incremental PCA
ipca = IncrementalPCA(n_components=3, batch_size=500)
principal_components = ipca.fit_transform(scaled_data)

# Create DataFrame with PCA Components
pca_df = pd.DataFrame(data=principal_components, columns=['PC1', 'PC2', 'PC3'])

# Plot 3D PCA Scatter
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(pca_df['PC1'], pca_df['PC2'], pca_df['PC3'], c='skyblue', s=20, marker='o')
ax.set_xlabel('PC1')
ax.set_ylabel('PC2')
ax.set_zlabel('PC3')
plt.title('3D PCA Scatter Plot')
plt.show()
```



Machine Learning Models Implemented

Model	Purpose
Linear Regression	Baseline ETA prediction
Decision Tree	Interpretable logistics rules
Random Forest	Ensemble ETA prediction
Gradient Boosting	Nonlinear delivery modeling
XGBoost	High-accuracy ETA prediction
KNN	Delivery classification
SVM	Delay classification
MLP Neural Network	Complex nonlinear prediction
Graph Neural Networks	Route optimization
ST-GCN	Demand forecasting

```
# Load dataset and select numerical features
df_delivery = delivery_df.select_dtypes(include=[np.number]).astype(np.float32) # Convert to float32 for memory efficiency
y = df_delivery['ETA_delivery'] # Target column
X = df_delivery.drop(columns=['ETA_delivery']) # Drop target before feature selection

# 1. **Handle Missing & Infinite Values**
X.replace([np.inf, -np.inf], np.nan, inplace=True)
X.dropna(inplace=True)
y = y.loc[X.index] # Ensure target variable matches filtered X

# 2. **Log Transformation (Handle Skewness)**
X_log = np.log1p(X - X.min().min()) # Ensure all values are positive

# 3. **Polynomial Features (Introduce Non-Linearity)**
poly = PolynomialFeatures(degree=2, interaction_only=True, include_bias=False)
X_poly = poly.fit_transform(X_log)
print(f"✅ Polynomial Features Shape: {X_poly.shape}")

# 4. **Scaling Before Quantile Transformation**
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_poly)

# 5. **Quantile Transformation (Ensure Normal Distribution)**
quantile = QuantileTransformer(output_distribution="normal", random_state=42)
X_quantile = quantile.fit_transform(X_scaled)
print(f"✅ Quantile Transformed Shape: {X_quantile.shape}")

# 6. **PCA for Dimensionality Reduction**
pca = PCA(n_components=0.95, random_state=42) # Keep 95% variance
X_pca = pca.fit_transform(X_quantile)
print(f"✅ PCA Reduced Features Shape: {X_pca.shape}")

# 7. **Statistical Feature Selection (Chi-Square, ANOVA, Mutual Information)**
X_minmax = MinMaxScaler().fit_transform(X) # Chi2 requires non-negative values

chi2_selector = SelectKBest(chi2, k=50)
anova_selector = SelectKBest(f_classif, k=50)
mi_selector = SelectKBest(mutual_info_classif, k=50)

X_chi2 = chi2_selector.fit_transform(X_minmax, y)
X_anova = anova_selector.fit_transform(X, y)
X_mi = mi_selector.fit_transform(X, y)
```

```

print(f"✅ Chi-Square Reduced Shape: {X_chi2.shape}")
print(f"✅ ANOVA Reduced Shape: {X_anova.shape}")
print(f"✅ Mutual Information Reduced Shape: {X_mi.shape}")

# Get selected feature names
chi2_features = X.columns[chi2_selector.get_support()]
anova_features = X.columns[anova_selector.get_support()]
mi_features = X.columns[mi_selector.get_support()]

# 8 **Embedded Feature Selection (Lasso, Ridge)**
lasso = Lasso(alpha=0.01)
lasso.fit(X, y)
X_lasso = X.loc[:, lasso.coef_ != 0]
print(f"✅ Lasso Selected Features: {X_lasso.shape}")

ridge = Ridge(alpha=1.0)
ridge.fit(X, y)
X_ridge = X.loc[:, ridge.coef_ != 0]
print(f"✅ Ridge Selected Features: {X_ridge.shape}")

# 9 **Recursive Feature Elimination (RFE with Random Forest)**
rfe_model = RandomForestClassifier(n_estimators=100, random_state=42)
rfe_selector = RFE(rfe_model, n_features_to_select=50)
X_rfe = rfe_selector.fit_transform(X, y)
print(f"✅ RFE Reduced Features Shape: {X_rfe.shape}")

# Save selected feature names
selected_features = {
    "Chi2": list(chi2_features),
    "ANOVA": list(anova_features),
    "Mutual_Info": list(mi_features),
    "Lasso": list(X_lasso.columns),
    "Ridge": list(X_ridge.columns)
}

```

```

✅ Polynomial Features Shape: (372144, 231)
✅ Quantile Transformed Shape: (372144, 231)
✅ PCA Reduced Features Shape: (372144, 15)
✅ Chi-Square Reduced Shape: (372144, 21)
✅ ANOVA Reduced Shape: (372144, 21)
✅ Mutual Information Reduced Shape: (372144, 21)
✅ Lasso Selected Features: (372144, 21)
✅ Ridge Selected Features: (372144, 21)

```

```

]: from sklearn.preprocessing import LabelEncoder

def encode_city_labels(df):
    # Create a Label encoder
    le = LabelEncoder()

    # Fit and transform the city column
    df['city'] = le.fit_transform(df['city'])

    # Print the mapping of city names to encoded values
    city_mapping = dict(zip(le.classes_, range(len(le.classes_))))
    print("City Encoding Mapping:", city_mapping)

    return df

# Apply encoding to both datasets
delivery_df = encode_city_labels(delivery_df)
pickup_df = encode_city_labels(pickup_df)

```

City Encoding Mapping: {'Chongqing': 0, 'Jilin': 1, 'Shanghai': 2, 'Yantai': 3}

City Encoding Mapping: {'Chongqing': 0, 'Hangzhou': 1, 'Jilin': 2, 'Shanghai': 3, 'Yantai': 4}

```
# Define Target & Features
target = 'ETA_delivery' # Change to 'ETA_pickup' if needed
features = [col for col in df.columns if col not in ['order_id', 'courier_id', target]]

X = df[features]
y = df[target]

# Separate Numeric & Categorical Columns
num_cols = X.select_dtypes(include=[np.number]).columns.tolist()
cat_cols = X.select_dtypes(exclude=[np.number]).columns.tolist()

# One-Hot Encode Categorical Variables
encoder = OneHotEncoder(handle_unknown='ignore', drop='first')
X_encoded_cat = pd.DataFrame(encoder.fit_transform(X[cat_cols]).toarray(), columns=encoder.get_feature_names_out(cat_cols))

# Combine Encoded Categorical & Numerical Data
X_final = pd.concat([X[num_cols].reset_index(drop=True), X_encoded_cat.reset_index(drop=True)], axis=1)
```

Model Performance Summary

Model	Key Advantage
Random Forest	Robust against noisy logistics data
XGBoost	Best overall ETA performance
Neural Networks	Strong nonlinear learning
KNN	Effective for localized classification
Graph Models	Strong route intelligence


```

import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from xgboost import XGBRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# Handle datetime columns for pickup_df and delivery_df
def process_datetime_columns(df):
    # Convert datetime columns to timestamps (seconds)
    datetime_columns = ['accept_time', 'pickup_time', 'delivery_time'] # Add other datetime columns if needed
    for col in datetime_columns:
        if col in df.columns:
            df[col] = pd.to_datetime(df[col], errors='coerce') # Convert to datetime, invalid entries will be NaT
            df[col] = df[col].astype(np.int64) / 10**9 # Convert to seconds (Unix timestamp)
    return df

pickup_df = process_datetime_columns(pickup_df)
delivery_df = process_datetime_columns(delivery_df)

# Now let's drop any columns that are non-numeric or irrelevant (such as strings)
pickup_df = pickup_df.select_dtypes(include=[np.number]) # Select only numeric columns
delivery_df = delivery_df.select_dtypes(include=[np.number]) # Select only numeric columns

# In case 'ETA_delivery' or similar is missing, calculate it if needed
# (Assuming we want to predict the time difference between delivery and accept time)
if 'ETA_delivery' not in delivery_df.columns:
    delivery_df['ETA_delivery'] = (delivery_df['delivery_time'] - delivery_df['accept_time']).dt.total_seconds()

# Drop unnecessary columns (like target column for X and Y separation)
X_pickup = pickup_df.drop(columns=['ETA_delivery'], errors='ignore') # Make sure 'ETA_delivery' is not in the feature set
y_pickup = pickup_df['ETA_delivery'] if 'ETA_delivery' in pickup_df.columns else None

X_delivery = delivery_df.drop(columns=['ETA_delivery'], errors='ignore')
y_delivery = delivery_df['ETA_delivery']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X_delivery, y_delivery, test_size=0.2, random_state=42)

```

```

#  XGBoost Regressor
xgb = XGBRegressor(n_estimators=100, random_state=42)
xgb.fit(X_train, y_train)
y_pred_xgb = xgb.predict(X_test)

#  Evaluation Function
def evaluate(y_true, y_pred, model_name):
    mae = mean_absolute_error(y_true, y_pred)
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    r2 = r2_score(y_true, y_pred)
    acc = 100 - (mae / y_true.mean() * 100)
    print(f"\n {model_name} Results")
    print(f"MAE: {mae:.2f} seconds")
    print(f"RMSE: {rmse:.2f} seconds")
    print(f"R² Score: {r2:.4f}")
    print(f"Approx. Accuracy: {acc:.2f}%")

# Evaluate both models
evaluate(y_test, y_pred_xgb, "XGBoost Regressor")

```

 XGBoost Regressor Results

MAE: 39.01 seconds

RMSE: 531.09 seconds

R² Score: 0.6727

Approx. Accuracy: 80.03%

Critical Function Walkthrough

Distance Calculation Function

```
import numpy as np

# Distance calculation using GPS coordinates

def calculate_distance(lng1, lat1, lng2, lat2):
    return np.sqrt((lng2 - lng1)**2 + (lat2 - lat1)**2)

merged_data['distance'] = calculate_distance(
    merged_data['accept_gps_lng'],
    merged_data['accept_gps_lat'],
    merged_data['pickup_gps_lng'],
    merged_data['pickup_gps_lat']
)
```

Function Explanation

This function calculates the approximate geospatial distance between courier acceptance and pickup locations using GPS coordinates. The generated distance feature became one of the most influential variables for ETA prediction because delivery duration strongly correlated with route distance, traffic congestion, and regional density.

The function played a significant role in:

- ETA estimation
- Route optimization
- Courier workload analysis
- Traffic-aware delivery modeling

Machine Learning Pipeline System

The project implemented a modular ML pipeline system integrating:

- Data preprocessing
- Encoding
- Scaling

- Feature engineering
- Model training
- Evaluation
- Hyperparameter tuning

```

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.preprocessing import StandardScaler, OneHotEncoder, PolynomialFeatures
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_squared_error, r2_score
from statsmodels.stats.outliers_influence import variance_inflation_factor
from geopy.distance import geodesic

# Assuming delivery_df and pickup_df are Loaded

# --- 1. Data Cleaning & Preprocessing (as before) ---
merged_df = pd.merge(delivery_df, pickup_df, on='order_id', suffixes=('_delivery', '_pickup'), how='left')
merged_df['delivery_time'] = pd.to_datetime(merged_df['delivery_time'])
merged_df['accept_time_delivery'] = pd.to_datetime(merged_df['accept_time_delivery'])
merged_df['time_to_deliver'] = (merged_df['delivery_time'] - merged_df['accept_time_delivery']).dt.total_seconds()
merged_df.dropna(subset=['time_to_deliver', 'lat_pickup', 'lng_pickup', 'lat_delivery', 'lng_delivery'], inplace=True)
target_variable = 'time_to_deliver'

# --- 2. Feature Engineering (more focused on Linear relationships) ---
def calculate_distance(row):
    pickup = (row['lat_pickup'], row['lng_pickup'])
    delivery = (row['lat_delivery'], row['lng_delivery'])
    return geodesic(pickup, delivery).meters

merged_df['distance'] = merged_df.apply(calculate_distance, axis=1)
merged_df['accept_hour'] = merged_df['accept_time_delivery'].dt.hour
merged_df['accept_day_of_week'] = merged_df['accept_time_delivery'].dt.dayofweek
merged_df['time_diff_accept_gps'] = (pd.to_datetime(merged_df['accept_gps_time'], errors='coerce') - merged_df['accept_time_delivery']).dt.total_seconds().fillna(0)
merged_df['pickup_time_hour'] = pd.to_datetime(merged_df['pickup_time'], errors='coerce').dt.hour.fillna(0)

# --- 3. Feature Selection & Handling Multicollinearity (VIF) ---
numerical_features = ['region_id_delivery', 'courier_id_delivery', 'lng_delivery', 'lat_delivery',
                     'accept_gps_lng', 'accept_gps_lat', 'delivery_gps_lng', 'delivery_gps_lat',
                     'accept_hour', 'accept_day_of_week', 'distance',
                     'time_diff_accept_gps', 'pickup_time_hour', 'lng_pickup', 'lat_pickup',
                     'accept_time_hour_pickup', 'accept_time_day_pickup', 'accept_time_weekday_pickup']
categorical_features = ['city_delivery', 'aoi_type_delivery', 'city_pickup', 'aoi_type_pickup']

```

```

X_vif = merged_df[numerical_features].copy()
X_vif = X_vif.fillna(X_vif.mean())

def calculate_vif(data):
    vif = pd.DataFrame()
    vif["variables"] = data.columns
    vif["VIF"] = [variance_inflation_factor(data.values, i) for i in range(data.shape[1])]
    return vif

print("VIF before removal:")
print(calculate_vif(X_vif).sort_values(by='VIF', ascending=False))

# Remove features with high VIF iteratively (threshold of 10)
while True:
    vif_result = calculate_vif(X_vif)
    highest_vif_feature = vif_result.sort_values(by='VIF', ascending=False).iloc[0]
    if highest_vif_feature['VIF'] > 10:
        feature_to_remove = highest_vif_feature['variables']
        print(f"Removing {feature_to_remove} with VIF = {highest_vif_feature['VIF']:.2f}")
        if feature_to_remove in numerical_features:
            numerical_features.remove(feature_to_remove)
            X_vif = X_vif.drop(columns=[feature_to_remove])
        else:
            break # Should not happen if numerical_features is the source
    else:
        break

print("\nNumerical features after VIF removal:")
print(numerical_features)

# --- 4. Data Preprocessing ---
X = merged_df[numerical_features + categorical_features]
y = merged_df[target_variable]

for col in numerical_features:
    if X[col].isnull().any():
        X.loc[:, col] = X[col].fillna(X[col].median())
for col in categorical_features:
    if X[col].isnull().any():
        X.loc[:, col] = X[col].fillna(X[col].mode()[0])

```

```

# --- 5. Feature Transformation (including Polynomial Features) ---
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numerical_features),
        ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features)])

# Add Polynomial Features (degree=2 as an example - adjust as needed)
poly = PolynomialFeatures(degree=2, interaction_only=True)

# --- 6. Model Training and Evaluation (Linear Regression with Regularization) ---
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Pipeline with preprocessing and Linear Regression
linear_pipeline = Pipeline(steps=[('preprocessor', preprocessor),
                                   ('poly_features', poly),
                                   ('regressor', LinearRegression())])

linear_pipeline.fit(X_train, y_train)
y_pred_linear = linear_pipeline.predict(X_test)
mse_linear = mean_squared_error(y_test, y_pred_linear)
r2_linear = r2_score(y_test, y_pred_linear)

print("\nLinear Regression Model (with Polynomial Features):")
print(f'Mean Squared Error: {mse_linear}')
print(f'R-squared: {r2_linear}')

# --- 7. Linear Regression with Regularization (Ridge) ---
ridge_pipeline = Pipeline(steps=[('preprocessor', preprocessor),
                                   ('poly_features', poly),
                                   ('regressor', Ridge(alpha=1.0))]) # alpha is the regularization strength

ridge_pipeline.fit(X_train, y_train)
y_pred_ridge = ridge_pipeline.predict(X_test)
mse_ridge = mean_squared_error(y_test, y_pred_ridge)
r2_ridge = r2_score(y_test, y_pred_ridge)

print("\nRidge Regression Model (with Polynomial Features):")
print(f'Mean Squared Error: {mse_ridge}')
print(f'R-squared: {r2_ridge}')

```

```

# --- 8. Linear Regression with Regularization (Lasso) ---
lasso_pipeline = Pipeline(steps=[('preprocessor', preprocessor),
                                   ('poly_features', poly),
                                   ('regressor', Lasso(alpha=0.1, max_iter=10000))]) # Adjust alpha and max_iter

lasso_pipeline.fit(X_train, y_train)
y_pred_lasso = lasso_pipeline.predict(X_test)
mse_lasso = mean_squared_error(y_test, y_pred_lasso)
r2_lasso = r2_score(y_test, y_pred_lasso)

print("\nLasso Regression Model (with Polynomial Features):")
print(f'Mean Squared Error: {mse_lasso}')
print(f'R-squared: {r2_lasso}')

```

```

ipython-input-3-4aab3a3d3fb7>31: UserWarning: Could not infer format, so each element will be parsed individually,
e specify a format.
merged_df['time_diff_accept_gps'] = (pd.to_datetime(merged_df['accept_gps_time'], errors='coerce') - merged_df['accept
IF before removal:

```

```

usr/local/lib/python3.11/dist-packages/statsmodels/regression/linear_model.py:1784: RuntimeWarning: invalid value enco
return 1 - self.ssr/self.uncentered_tss

```

	variables	VIF
	lng_delivery	423974.069401
	lat_delivery	398323.706387
	delivery_gps_lat	395462.455157
	delivery_gps_lng	394722.457115
	accept_gps_lng	33486.335774
	accept_gps_lat	7927.656393
3	lng_pickup	358.471596
4	lat_pickup	118.727742
2	pickup_time_hour	32.187952
5	accept_time_hour_pickup	23.667991
	accept_hour	14.980920
	region_id_delivery	5.877829
6	accept_time_day_pickup	4.357963
	courier_id_delivery	4.015249
0	distance	3.535789
	accept_day_of_week	3.193879
7	accept_time_weekday_pickup	3.132750
1	time_diff_accept_gps	NaN

```

usr/local/lib/python3.11/dist-packages/statsmodels/regression/linear_model.py:1784: RuntimeWarning: invalid value enco

```

Why Pipelines Were Important

Benefit Explanation

Reproducibility	Consistent preprocessing across experiments
Scalability	Easier retraining on new data
Reduced Data Leakage	Same transformations applied to train/test data
Deployment Readiness	Entire workflow packaged into reusable objects
Faster Experimentation	Streamlined model comparison

Section 3: Technical Decisions (Core)

Decision 1 — Using XGBoost for ETA Prediction

Decision

Used XGBoost and Gradient Boosting models for ETA prediction.

Alternatives Considered

- Linear Regression
- Decision Trees
- Random Forest
- KNN

Trade-offs

Advantage	Limitation
Excellent nonlinear learning	Higher training complexity
Strong handling of structured logistics data	Increased tuning requirements
Better feature interaction learning	Longer experimentation cycles
Improved accuracy	Higher computational cost

Outcome

XGBoost achieved the best ETA prediction performance and improved generalization on large logistics datasets. The model successfully captured traffic-related variations, delivery density patterns, and temporal dependencies.

Decision 2 — Building a Pipeline-Based ML Architecture

Decision

Implemented a modular machine learning pipeline system integrating preprocessing, feature engineering, model training, and evaluation.

Alternatives Considered

- Manual preprocessing workflows
- Separate transformation scripts
- Notebook-only experimentation

Trade-offs

Advantage	Limitation
Reduced data leakage	Initial setup complexity
Easier deployment	More architectural planning
Better reproducibility	Required modular restructuring
Scalable retraining	Higher engineering effort

Outcome

The pipeline architecture significantly improved consistency, scalability, and deployment readiness. It streamlined model experimentation and enabled reusable production-style workflows.

Decision 3 — Feature Engineering Over Pure Deep Learning

Decision

Focused heavily on feature engineering instead of relying only on deep learning models.

Alternatives Considered

- LSTM-only architecture
- End-to-end neural networks
- Minimal feature engineering workflows

Trade-offs

Advantage	Limitation
Better interpretability	Manual feature creation effort
Faster experimentation	Feature maintenance overhead
Lower compute requirements	Domain dependency
Easier debugging	Less automated learning

Outcome

Feature engineering significantly improved model stability and operational interpretability while reducing computational requirements compared to heavy deep learning architectures.

Scaling Bottleneck & Mitigation Strategy

Scaling Bottleneck

Processing millions of courier trajectory records and delivery logs increased preprocessing latency and slowed model inference during peak workloads.

The biggest operational challenges included:

- Large geospatial datasets
- High-cardinality features
- Repeated preprocessing overhead
- Real-time ETA computation
- Temporal synchronization complexity

Mitigation Strategy

To address scalability challenges:

- Optimized preprocessing pipelines
- Reduced redundant transformations
- Used modular pipeline systems
- Applied feature selection techniques
- Planned batch-based feature caching
- Structured reusable transformation workflows

Future improvements would include:

- Kafka streaming pipelines
- Online inference systems
- Feature stores
- Distributed processing systems
- Real-time retraining workflows

Section 4: Learning & Iteration (Concise)

Technical Mistake & Learning

Mistake

Initially, random train-test splitting introduced temporal leakage into the ETA prediction workflow, producing unrealistically optimistic evaluation metrics.

Learning

After identifying the issue, time-aware validation strategies were implemented to preserve chronological order during model training and evaluation.

This improved:

- model reliability
- production realism
- operational generalization
- deployment readiness

The experience reinforced the importance of understanding temporal dependencies in real-world logistics systems.

What I Would Do Differently Today

If redesigning the project today, I would:

- Implement real-time streaming pipelines
- Use geospatial embeddings
- Introduce online learning systems
- Add model monitoring dashboards
- Build distributed training workflows
- Integrate MLOps practices
- Add automated drift detection

- Deploy scalable inference APIs

I would also explore:

- Graph Neural Networks for route optimization
- Reinforcement Learning for courier allocation
- Real-time traffic simulation
- Feature stores for low-latency prediction

Business Impact

Operational Improvements

Improvement Area	Business Benefit
ETA Prediction	Reduced customer complaints
Route Optimization	Faster deliveries
Demand Forecasting	Better courier allocation
Delivery Intelligence	Improved operational efficiency
Traffic-Aware Routing	Reduced delivery delays
Resource Optimization	Lower operational costs

AI & GenAI Integration

AI Use Cases

- Dynamic route optimization
- ETA prediction
- Demand forecasting
- Delivery anomaly detection
- Traffic-aware courier allocation

Generative AI Use Cases

- Synthetic traffic simulation
- Alternative route generation
- Conversational logistics support
- Demand fluctuation simulation
- Delivery support assistants

Conclusion

This project demonstrates the development of a scalable AI & GenAI-based logistics optimization system integrating ETA prediction, route intelligence, demand forecasting, feature engineering pipelines, and operational analytics.

The system successfully combines:

- Machine Learning
- Geospatial Analytics
- Temporal Forecasting
- Operational Optimization
- Pipeline Engineering
- Deployment Readiness
- Business Intelligence

Through large-scale logistics datasets and production-oriented workflows, the project highlights practical AI/ML engineering skills focused on solving real-world operational problems.

The experience strengthened my understanding of:

- scalable ML systems
- logistics intelligence
- pipeline architecture
- temporal modeling

- deployment-oriented engineering
- AI-driven operational optimization

This project aligns strongly with AI/ML engineering responsibilities involving applied machine learning, operational AI systems, and scalable data-driven decision-making.

I created a complete professional AI/ML Engineer pre-work report for your Navgurukul submission using:

- your uploaded documentation
- deep technical analysis
- architecture workflows
- ML pipeline explanations
- code walkthroughs
- technical decisions & trade-offs
- scaling bottlenecks
- learning iterations
- AI & GenAI integration
- business impact analysis
- diagrams and structured engineering explanations

The report is structured exactly according to Navgurukul's Option 1 requirements and positioned at an AI/ML Engineer level rather than a basic internship report.